Step-level Optimization for Efficient Computer-use Agents

Jinbiao Wei, Kangqi Ni, Yilun Zhao, Guo Gan, Arman Cohan

Paper version: arXiv 2604.27151v1 · CUA Radar Reel poster, not made or endorsed by the authors

Problem

> *Most systems invoke large multimodal models at nearly every interaction step.*

Progress stalls (loops, repeats) and **silent drift** (plausible but off-goal) concentrate the cost.

Motivation: spend big compute only at risky moments

1. **Routine vs risky.** Errors cluster at a few moments.
2. **Two signatures.** Failed runs take 2.8× more steps, repeat actions.
3. **Sparse oversight.** Step checks = costly; end-only = late.

Evaluation setup

OSWorld (desktop) · WebArena-Verified

4 small policies · 4 large models

- **All online** execution checks; OpenRouter costs.
- **Monitors:** 300 trajectories, GPT-5.2 labels ×5 runs; no seeds/CIs.

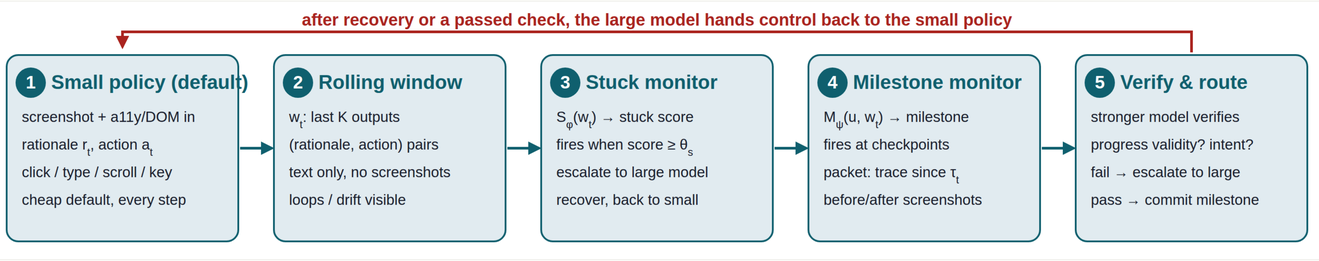
Our read (CUA Radar)

63	★★	transfer
score of 100	band	evidence level

- **Reports:** online OSWorld + WebArena; periodic-k baseline; cost/latency; ablation.
- **Not reported:** seeds/variance; held-out envs; code; human labels (AP4).
- Pattern C18; authors' numbers.

Method: StepWise, an event-driven step-level cascade

- **Sees / acts:** screenshot (+ optional a11y/DOM) in; click, type, scroll, hotkey out — desktop and web.
- **Cascade:** a small policy acts by default and escalates to the large model only when a monitor fires; the handoff re-serializes the small model's history.
- **Signal:** two ModernBERT monitors score rolling text windows of (rationale, action) pairs — no screenshots, cheap enough to run every step.

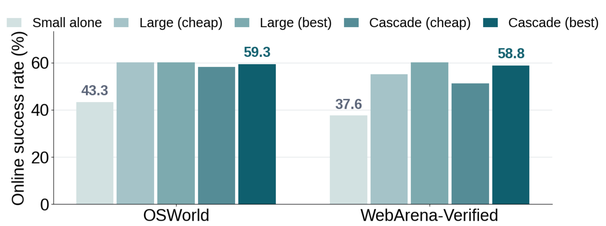


Our redraw (not the paper's figure): default small policy, two text-only monitors, escalation handoff.

$$p_t^{\text{stuck}} = S_\phi(w_t), \qquad p_t^{\text{mile}} = M_\psi(u, w_t)$$

Two monitors over the window wt of the last K outputs; only Mψ also sees the task u.

Results: online success rate (%)



Tables 1-2, redrawn. Standalone models vs step-level cascades; all runs online and execution-checked.

59.3 vs 60.1 (OSW), 58.8 vs 60.1 (WA) at up to 61.4% lower cost

SO WHAT → *Small: EvoCUA-8B / gpt-oss-20b · Large: Kimi K2.5 / GPT-5 mini / GPT-5.2 · best cascade per benchmark.*

Ablation: which detector fires

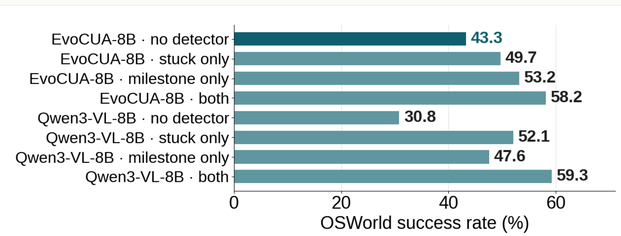


Figure 3, redrawn from its printed bar labels (large model: Kimi K2.5 / GPT-5.2).

SO WHAT → *Either detector alone beats none; both together is best on every pair. WebArena mirrors it: gpt-oss-20b 37.6 without detectors → 57.8 with both; AgentTrek-32B 30.3 → 58.8. The signals split the failure space: stuck detection lifts looping policies (Qwen3-VL-8B 30.8 → 52.1 stuck-only); milestone checks catch drift (gpt-oss-20b 44.9 stuck-only vs 54.7 milestone-only).*

Headline Numbers

59.3%
OSWorld cascade success
vs 60.1% always-large, $0.078/task

74.6% max cost reduction

45.8% max latency cut

$0.051 per-task, EvoCUA+Kimi

The paper's own controls

CONTROL	RESULT	REFERENCE
Stuck detector	93.9% · 91.5 F1	GPT-5.2 92.7 F1
Milestone detector	94.1% · 62.0 F1	GPT-5.2 77.1 F1
Periodic-k (OSW)	55.1% · $0.07	event 58.2 · $0.05
Periodic-k (WA)	52.5% · $0.24	event 58.8 · $0.21

Detectors vs GPT-5.2 teacher labels (AP4).

Takeaway

Spend large-model compute only on stuck events and milestone checks: near always-large success at 74.6% lower cost, 45.8% lower latency (max).

stuck F1 91.5 · milestone F1 62.0 · $0.051 per task · 2 monitors, 149M each

Scan to Read

Paper · Code